

REPORTE DE PRÁCTICA: ESTRUCTURAS SELECTIVAS Y CONDICIONALES

Asignatura: Programación Estructurada
Estudiante: Aldo Ezequiel Rodriguez Godoy
Código: 224097617
Sección: D04
Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha: 13 de febrero del 2026

INTRODUCCIÓN

En el desarrollo de software bajo el paradigma estructurado, los programas ejecutan instrucciones de manera secuencial por defecto. Sin embargo, para resolver problemas del mundo real se requiere que el flujo del algoritmo tome decisiones basadas en estados o datos cambiantes. Aquí es donde entran las estructuras selectivas o condicionales (como el `if` y el `else`), las cuales evalúan expresiones lógicas que devuelven un valor verdadero o falso para bifurcar el camino que seguirá el hilo de ejecución.

En esta actividad nos enfocamos en el uso de estructuras condicionales anidadas y operadores lógicos. Los operadores lógicos, tales como la conjunción (AND lógico representado por `&&`) y la disyunción (OR lógico representado por `||`), permiten combinar múltiples comparaciones en una sola sentencia analítica. Evaluamos esto aplicándolo a un problema clásico: determinar si un número entero provisto por el usuario se encuentra dentro de un rango específico (del 1 al 10) y validar de forma matemática y directa si dicho número se clasifica como un número primo.

OBJETIVO

Objetivo General

Construir e implementar un algoritmo funcional en lenguaje C que haga uso de estructuras condicionales anidadas e interconexiones lógicas para evaluar la pertenencia de un número de entrada a un rango numérico preestablecido, discriminando con precisión si el valor corresponde a un número primo.

Objetivos Particulares

- Implementar validaciones de rango cerrado utilizando operadores relacionales de desigualdad (`>` y `<=`) mediante la conjunción lógica `&&`.
- Utilizar sentencias condicionales anidadas para segmentar las etapas de validación del programa (filtrado de rango primero y evaluación matemática después).
- Dominar la aplicación de la disyunción lógica `||` para verificar la igualdad múltiple de una variable contra constantes determinadas de manera compacta y eficiente.

DESARROLLO

Para construir esta actividad, diseñé el código enfocándose en un flujo lógico estructurado que evitara procesos innecesarios. El desarrollo del algoritmo consta de los siguientes pasos procedimentales:

1. Captura del Dato de Entrada

Definí una variable local de tipo entero llamada `nm` para almacenar el número de prueba. Mediante la instrucción `printf` le solicité al usuario ingresar un valor y utilicé `scanf` con el formateador `%i` para leerlo desde el teclado de forma directa.

2. Validación de Rango Mediante Filtro Inicial

Antes de calcular si el número es primo, implementé una estructura condicional `if` que valida si el número pertenece al rango permitido de la actividad (mayor a 1 y menor o igual a 10). Usé el operador lógico `&&` para juntar ambas restricciones en una sola línea. Si el número no cumple con este filtro, el flujo salta de inmediato al bloque `else` externo, el cual imprime que el dato está fuera de rango, impidiendo que se ejecuten evaluaciones innecesarias.

3. Evaluación de Números Primos Mediante Disyunción

Una vez que garantizamos que el número está en el rango correcto, el programa entra a un segundo `if` anidado. Como el rango es muy pequeño (del 2 al 10), opté por una solución de comparación directa muy eficiente: utilicé el operador lógico OR (`||`) para comprobar si el valor es exactamente igual a 2, 3, 5 o 7, que son los únicos números primos existentes en ese intervalo. Si la variable coincide con cualquiera de ellos, el programa imprime que es primo; de lo contrario, se ejecuta el `else` interno indicando que no lo es.

El código fuente final del programa quedó de la siguiente manera:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int nm;
    printf("ingresa el numero\n");
    scanf("%i", &nm);

    // Filtro externo: Validacion de rango entre 1 y 10 (excluyendo el 1)
    if (nm>1 && nm<=10){
        // Filtro anidado: Verificacion directa de los numeros primos del rango
        if (nm==1|nm==2|nm==3|nm==5|nm==7){

            printf ("es primo");

        }
        else {
            printf ("el numero no es primo");
        }
    }
    else
    {
        printf ("el numero esta fuera de rango ");
    }

    return 0;
}

```

CONCLUSIÓN

El desarrollo de esta práctica me sirvió mucho para comprender cómo estructurar la lógica de toma de decisiones en un programa de forma ordenada. Al principio del ejercicio se puede pensar que resolver un problema implica usar fórmulas matemáticas complejas para calcular los primos, pero al analizar las limitaciones y el alcance de la actividad (un rango fijo del 1 al 10), aprendí que el uso estratégico de operadores lógicos como el OR (|) permite simplificar el código drásticamente y hacerlo más rápido de ejecutar.

Lo que considero más valioso de este ejercicio fue trabajar con la anidación de las sentencias condicionales. Pude notar la gran importancia que tiene filtrar los datos desde un inicio; colocar el control de rango en el if externo protege al programa de trabajar con valores incorrectos. Esta mentalidad de validar primero los datos de entrada antes de procesar cualquier algoritmo es una excelente práctica que me ayudará a diseñar software más seguro y robusto en los siguientes temas de la materia.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Joyanes Aguilar, L. (2008). *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos* (4ta ed.). McGraw-Hill.